BST vs AVL Tree

A Comparison Report — Data Structures Assignment

Cairo University | Faculty of Computers and Artificial Intelligence | Marwan | 2025

1. Differences Observed

1.1 Random IDs — Case 1

With 20 books inserted in random order, both trees performed similarly. The BST reached a height of 6 and the AVL a height of 5 — just one level apart. Search steps were also close (4 steps each for ID 12 in this run).

This is expected: random IDs naturally spread insertions left and right, so the BST stays roughly balanced on its own without any rotations.

1.2 Sorted IDs — Case 2

Sorted insertion is where the gap becomes dramatic. When books are inserted with IDs 1, 2, 3, … 20 in order, every new node ends up as the right child of the previous one. The BST degenerates into a straight right-leaning chain — height 19, nearly equal to the number of nodes. Searching for ID 12 required 11 steps.

The AVL Tree maintained a height of only 5. Each time the balance factor of any node reached +2 or -2, it triggered one of the four rotation types (LL, RR, LR, RL) to redistribute nodes and restore balance. Searching for the same ID 12 took only 3 steps.

1.3 Summary Table

Metric	BST — Random	AVL — Random	BST — Sorted	AVL — Sorted
Height	6	5	19	5
Search Steps (ID 12)	4	4	11	3
Balancing	None	Automatic	None	Automatic

Key takeaway: for random data the difference is minor. For sorted (or nearly sorted) data, BST height grows linearly — O(n) — while AVL is always O(log n). With 20 books the gap is already 19 vs 5; with 1,000 books it would be ~1,000 vs ~10.

2. Why Does This Happen?

A BST has no self-balancing mechanism. Insertion always follows the same rule — go right if larger, go left if smaller — with no regard for the resulting shape. When data arrives in sorted order, every node is strictly larger than the one before it, so every insertion goes right. The result is a structure identical to a linked list, giving O(n) search in the worst case.

An AVL Tree extends BST by storing a balance factor at each node (height of left subtree minus height of right subtree). After every insertion or deletion, it checks this factor. If it exceeds ±1 anywhere, it applies one of four targeted rotations to restore balance:

- LL rotation — single right rotation for a left-heavy left subtree
- RR rotation — single left rotation for a right-heavy right subtree
- LR rotation — left rotation on child, then right rotation on parent
- RL rotation — right rotation on child, then left rotation on parent

These rotations are O(1) each, so they add negligible overhead per insertion while guaranteeing the tree height never exceeds $1.44 \cdot \log_2(n)$ — which rounds to O(log n) in practice.

---

3. When to Use BST vs AVL

Use a BST when:

- Data is inserted in a genuinely random order and you can reasonably expect the tree to stay balanced naturally
- Simplicity matters — BST code is much shorter and easier to debug, with no rotation logic to maintain
- The dataset is small, so a height difference of a few levels has no measurable impact
- Insertions and deletions are far more frequent than searches, and you want to avoid the extra cost of rebalancing on every modification
- You need a quick prototype or a learning implementation

Use an AVL Tree when:

- Data may arrive in sorted or near-sorted order — this is the exact scenario where BST performance collapses
- You need guaranteed O(log n) search time regardless of insertion order, such as in a library catalog or database index
- The dataset is large — the difference between height 19 and height 5 for 20 books scales to hundreds versus ~10 for 1,000 books
- Searches happen far more often than insertions, making the one-time rotation cost worthwhile
- Consistent, predictable worst-case performance is a requirement and you cannot afford O(n) degradation